Building and Attacking an ICS/SCADA Honeypot

Conpot over Modbus, SNMP and S7comm — live attack probes and automated log analysis

[Your Name] [1] Awez [Surname] [1]

[1][Course / Module — University]

October 24, 2026

Honeypot & attacker side — [Your Name] ▪ *Log analysis & dashboard* — Awez

Outline

1. Background
2. Architecture
3. Attacking the honeypot
4. Results
5. Log analysis & detection
6. Wrap-up

Why an ICS/SCADA honeypot?

- Industrial Control Systems (ICS/SCADA) run power, water and factory plants using legacy protocols — **Modbus**, **S7comm**, **SNMP** — that were built with *little or no authentication*.
- These devices are increasingly reachable from the network, and attackers actively scan for them (Shodan, automated ICS worms).
- A **honeypot** is a decoy device: it looks like a real PLC, has no real process behind it, and *logs everything an attacker does*.

Our goal (two-person project)

Stand up a realistic ICS honeypot, attack it ourselves over three protocols, and turn the raw honeypot log into structured events, risk scores and a dashboard.

Conpot — the honeypot engine

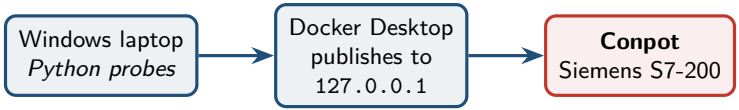
- Open-source, low-interaction ICS honeypot from the **Honeynet Project**.
- Emulates a **Siemens SIMATIC S7-200** PLC.
- Speaks **Modbus/TCP**, **S7comm**, **SNMP** and **HTTP** on one container.
- Answers with believable device identity, so a scanner thinks it found a real PLC — then records the interaction.

Division of work

Half A — deploy Conpot, attack it, capture `conpot.log`.

Half B — parse the log, detect attack patterns, build a dashboard.

System architecture



Attacker and honeypot run on one laptop: the Conpot **Docker** container publishes its ICS ports on `127.0.0.1`, so every probe targets localhost and reaches the container directly.

Service	**Laptop port**	**Conpot port**
Modbus	502	5020
S7comm	102	10201
SNMP (udp)	161	16100
HTTP	8080	8800

Attack methodology — three probes

All run from the Windows laptop against `127.0.0.1`; each is a small Python script that generates clean, labelled traffic for the log.

Modbus (pymodbus)

Read coils, discrete inputs, holding & input registers; then a **write** to a holding register (value 1234).

SNMP (pysnmp)

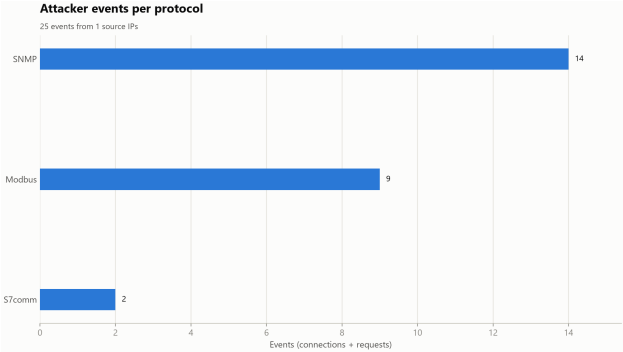
`GET` on system OIDs and a `WALK` of `1.3.6.1.2.1.1` (v2c, community `public`).

S7comm (snap7)

Establish an S7 session and request CPU identification (SZL) — what PLCScan does first.

The write + multi-protocol scan is deliberate: it is the behaviour of a real attacker moving from *reconnaissance* to *tampering*.

What the honeypot captured



- **25 attacker events** from a single source (`172.17.0.1`, the Docker bridge gateway = our laptop).
- **SNMP** dominated (14) — the full system-tree walk.
- **Modbus** (9): connect, four register reads, one write.
- **S7comm** (2): connection/identification probes.

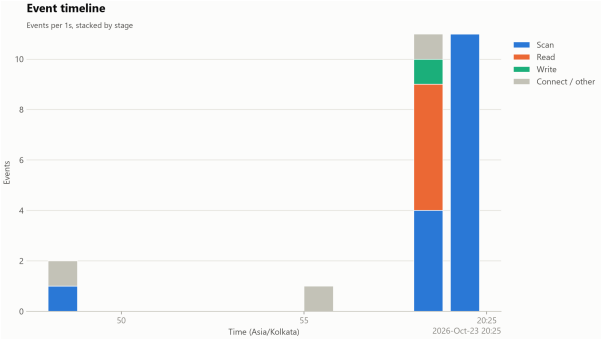
SNMP recon — the honeypot *is* a Siemens PLC

The SNMP probe returned a convincing fake identity, exactly what an ICS scanner looks for:

OID	Value returned by Conpot
sysDescr	Siemens, SIMATIC, S7-200
sysName	CP 443-1 EX40
sysContact	Siemens AG
sysLocation	Venus

Modbus reads answered with an `IllegalAddress` exception (the stock Conpot template defines no register values) — the connections and requests are still fully logged. The S7 session was established and logged.

From raw log to structured events

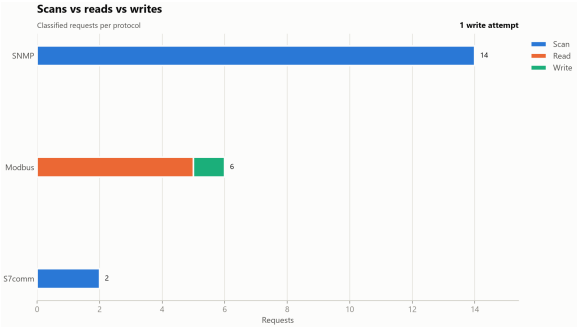


Parser (Half B). `conpot.log` → a pandas table, one row per event with:

- protocol, request type, function code
- action: **scan / read / write**
- source IP (+ private / Docker-gateway flags)

131 log lines → 25 clean events, verified by a **79-test** suite.

Attack-sequence detection & risk scoring



Per source IP, the detector scores risk from the attack *sequence*, multi-protocol scanning and any write attempts.

Our capture: `172.17.0.1`

Risk: **High**

Sequence: **scan** → **read** → **write**

Protocols: modbus, s7comm, snmp

Write attempt flagged

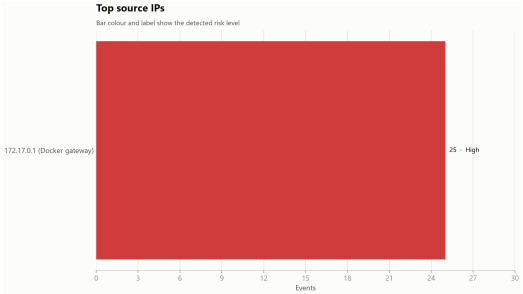
The full scan → read → write chain across three protocols is the classic recon-to-tamper pattern.

Live dashboard

A **Streamlit** dashboard (`dashboard/app.py`) reads the same log and shows, with auto-refresh:

- KPI tiles — total events, unique source IPs, write attempts.
- Risk-by-IP table (Low / Medium / **High**).
- Protocol mix and gateway share.

Run with: `streamlit run dashboard/app.py`



Top source IPs (from the real capture)

Ethics & scope

- Every probe targets **our own** honeypot container on **localhost** — a private, isolated university assignment.
- Conpot is purpose-built to be scanned and attacked; no third party is involved.
- No tool here is pointed at any host we do not own or have permission to test.

Why this is safe and useful

The honeypot has no real process to disrupt; the value is entirely in *observing and analysing* attacker behaviour in a controlled setting.

Conclusion & future work

What we built

- A working Conpot ICS honeypot (Siemens S7-200) exercised over Modbus, SNMP and S7comm.
- Our own captured attack traffic: 25 events, 3 protocols, scored **High** risk.
- An automated pipeline — parser, risk detection, charts and a live dashboard — with a 79-test suite.

Future work

- Expose the honeypot on an isolated public-facing network to capture *real* internet scans.
- Richer Conpot template (populated registers, more OIDs); add more ICS protocols.

Thank you

Questions?

Conpot ICS/SCADA honeypot — attack & analysis project